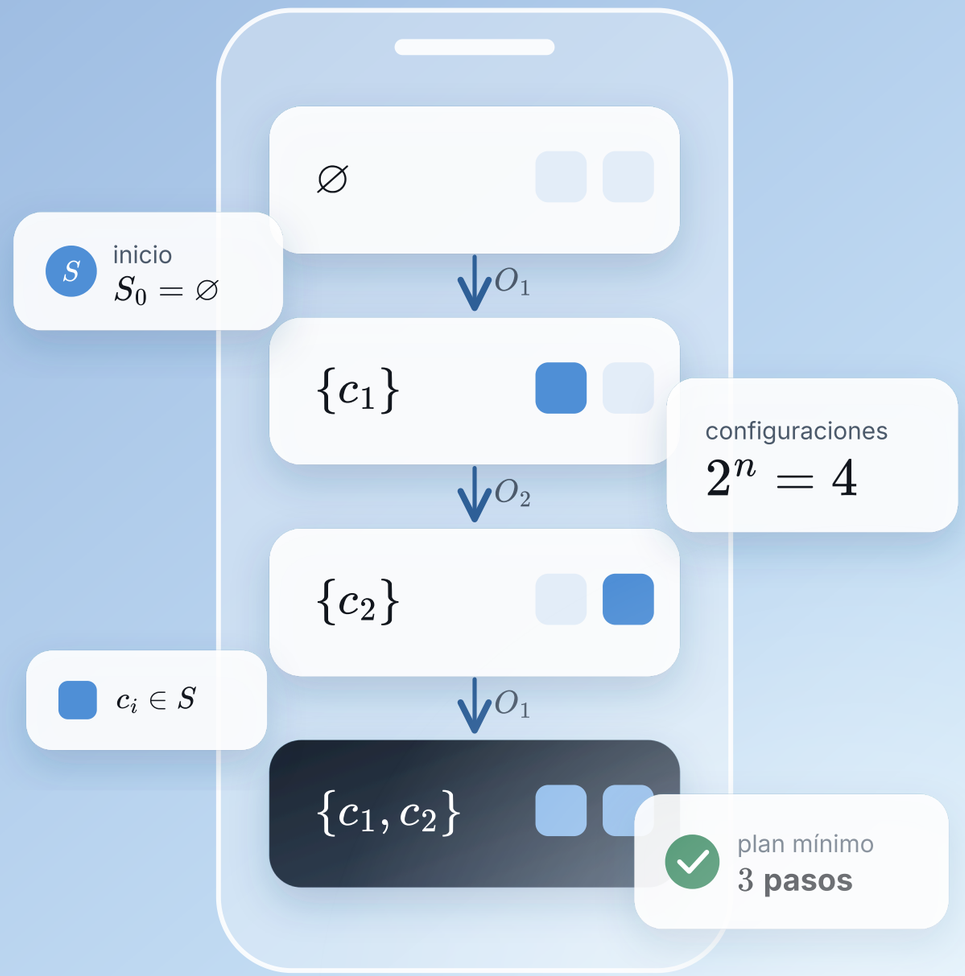


El problema de planeación

Planning: un problema PSPACE-completo

AG JR

Presentado por

Alan Alexzander González Murga

Jesús Antonio Romero Godoy

Definimos el problema, probamos que está en PSPACE con el algoritmo de Savitch y explicamos por qué es PSPACE-completo.

¿Existe un plan?

Partiendo de un estado inicial, ¿hay una secuencia de acciones permitidas que lleve a un estado objetivo?

Es el modelo detrás de la logística de una operación de socorro, de un robot que acomoda cajas o del rompecabezas de quince (Kleinberg y Tardos, 2005).



2^n

Configuraciones

posibles con n condiciones verdaderas o falsas.

$2^n - 1$

Pasos

puede medir el plan más corto en algunas instancias.



La clase PSPACE

Memoria polinomial

Problemas que se resuelven usando espacio polinomial en el tamaño de la entrada. Contiene a P y a NP, y está contenida en EXPTIME.

PSPACE-completo

$X \in$ PSPACE y todo $Y \in$ PSPACE cumple $Y \leq_P X$. El problema raíz es QSAT (Stockmeyer y Meyer, 1973).

El espacio se reutiliza

Un contador de n bits recorre 2^n valores en espacio $O(n)$: el espacio se recicla; el tiempo, no.

Condiciones y operadores



MODELO STRIPS PROPOSICIONAL

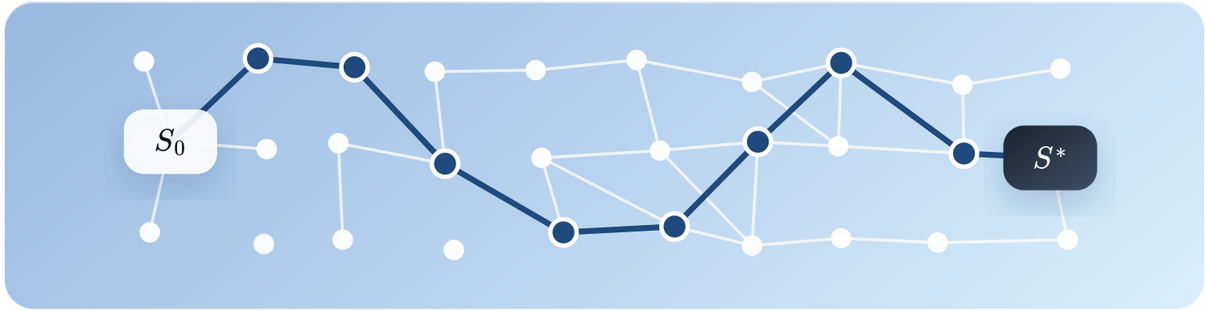
$$S' = (S \setminus D_i) \cup A_i$$

- **Condiciones**: $\mathcal{C} = \{c_1, \ldots, c_n\}$.
- **Configuración**: un subconjunto $S \subseteq \mathcal{C}$; hay 2^n posibles.
- **Operador** $O_i = (P_i, A_i, D_i)$: prerrequisitos, adiciones y eliminaciones. Es aplicable en S si $P_i \subseteq S$.

Fikes y Nilsson (1971); Kleinberg y Tardos (2005, p. 538).

El problema de decisión Planning

Equivale a buscar un camino en una grafica de 2^n configuraciones que está descrita de forma compacta y nunca se construye.



Entrada

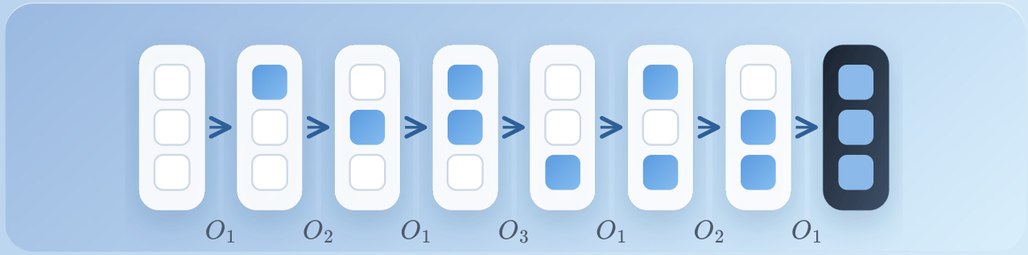
Condiciones $\mathcal{C}$, operadores (P_i, A_i, D_i), una configuración inicial S_0 y una meta S^*.

Pregunta

¿Existe una secuencia de operadores, cada uno aplicable, que lleve de S_0 **exactamente** a S^*?

$$S_0 \rightarrow O_{i_1} \rightarrow O_{i_2} \rightarrow \cdots \rightarrow S^*$$

Planes de longitud exponencial



$2^n - 1$

Plan mínimo del contador

Con n condiciones y n operadores, el plan más corto mide $2^n - 1$ pasos: cuenta en binario (Teorema 9.6).

$\leq 2^n - 1$

Cota superior

Si hay solución, hay una con a lo más $2^n - 1$ pasos: un camino simple en una grafica de 2^n nodos (Teorema 9.7).

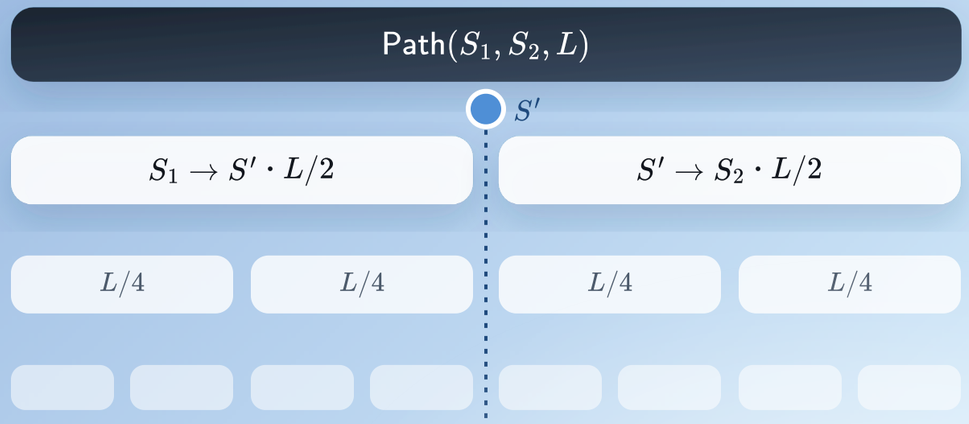
¿NP?

El plan no es certificado

Puede ser exponencial: ni siquiera cabe escribirlo en espacio polinomial. Hay que *decidir* sin verlo completo.

El procedimiento Path

Savitch (1970): divide y vencerás sobre la **longitud** del camino.



profundidad $\log L = n$ **· cada nivel guarda solo un bit**

Kleinberg y Tardos (2005, p. 542)

procedimiento Path(S_1, S_2, L)

  si $L = 1$ **entonces** devolver $S_1 = S_2$ o $\exists\, O : S_1 \xrightarrow{O} S_2$

  para cada S' (contador de n bits) **hacer**

    $x \leftarrow$ Path$(S_1, S', \lceil L/2 \rceil)$ — borrar todo salvo x

    $y \leftarrow$ Path$(S', S_2, \lceil L/2 \rceil)$ — borrar todo salvo y

    si x y y **entonces** devolver «sí»

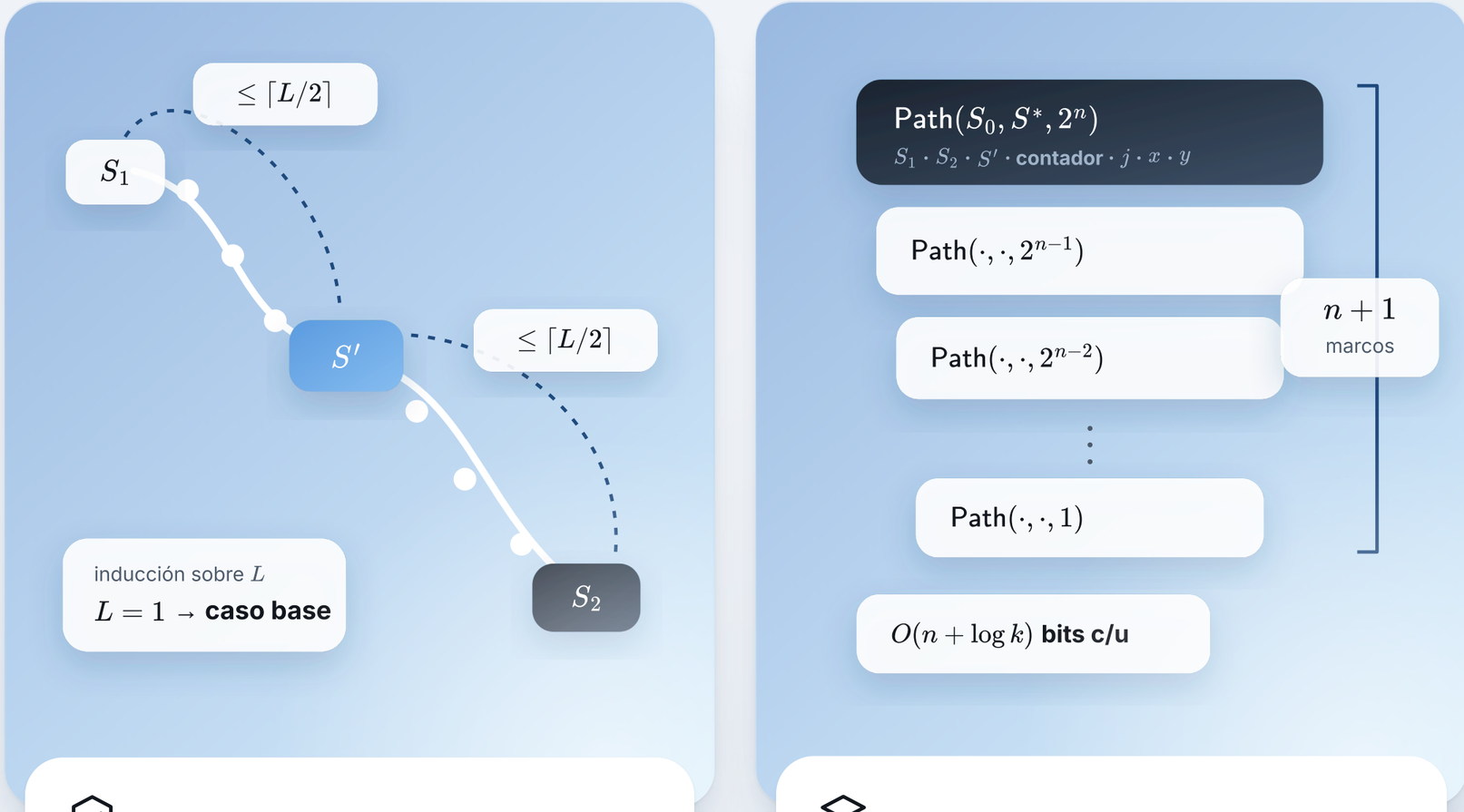
  devolver «no»

¿Hay camino de $\leq L$ pasos?

Prueba cada configuración S' como punto medio, resuelve las dos mitades con $\lceil L/2 \rceil$ y borra el trabajo intermedio: de cada mitad solo sobrevive un bit. El problema original es Path$(S_0, S^*, 2^n)$.

Planning está en PSPACE

Path$(S_0, S^*, 2^n)$ decide la instancia: es correcto y usa memoria polinomial.



Correctitud · Teorema 9.8

"Devuelve *sí* si y solo si existe una secuencia de a lo más L operadores. Por inducción sobre L, partiendo el camino en su punto medio."

Espacio · Teorema 9.8

"Solo hay una llamada activa por nivel: $n+1$ marcos de $O(n + \log k)$ bits, en total $O\big(n(n + \log k)\big)$."

Espacio a costa de tiempo

Path no guarda nada: tira el trabajo intermedio y lo recalcula. Es lo opuesto a la programación dinámica.

$N(1) = 1$ y $N(L) \le 1 + 2^{n+1}\, N(L/2)$, por lo que $N(2^n) \le 2 \cdot 2^{n(n+1)}$.



$2^{O(n^2)}$

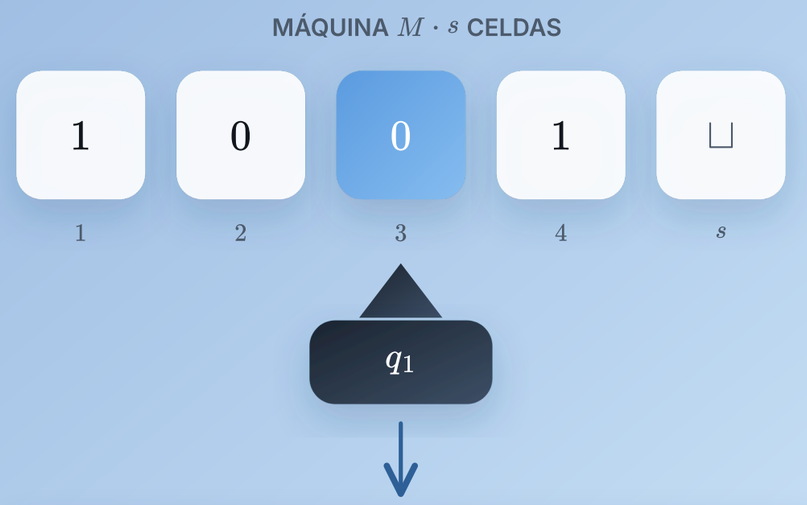
Tiempo

Exponencial: cada llamada cuesta tiempo polinomial, pero hay $2^{O(n^2)}$ llamadas.

$n + 1$

Marcos de pila

Cada uno de $O(n + \log k)$ bits: el espacio total es polinomial.



Reducción desde máquinas de Turing

Condiciones

Q_q (estado), H_i (cabeza en la celda i) y $S_{i,a}$ (la celda i contiene a). Son $|Q| + s(1 + |\Gamma|)$.

Operadores

Por cada transición $\delta(q, a) = (q', b, D)$ y cada celda i, un operador $O_{q,a,i}$ que cambia de estado, escribe b y mueve la cabeza.

Invariante

Siempre hay un estado, una cabeza y un símbolo por celda. M acepta w si y solo si existe un plan de S_0 a S^*.

Planning es PSPACE-completo



$\in$ PSPACE

Pertenencia

Path decide cualquier instancia con $O\big(n(n+\log k)\big)$ bits de memoria.

$\leq_P$

Dificultad

Todo $A \in$ PSPACE se reduce a Planning en tiempo polinomial (Bylander, 1994).

¿P?

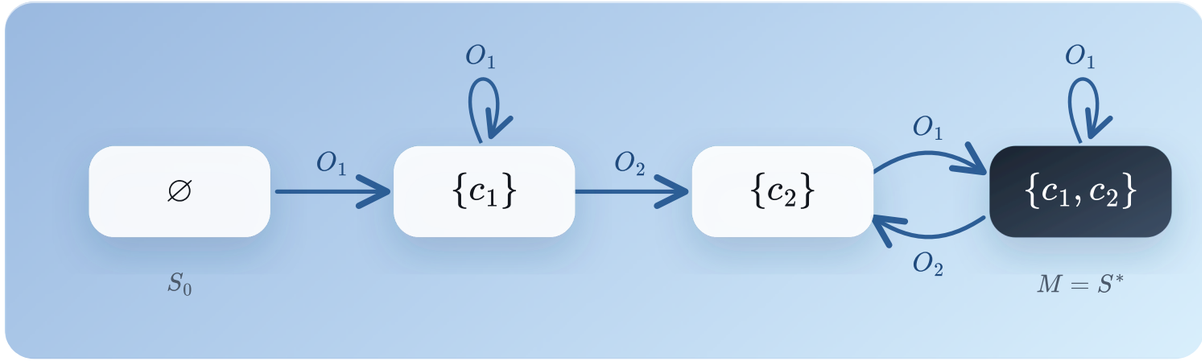
Consecuencia

Un algoritmo de tiempo polinomial para Planning implicaría P $=$ PSPACE, algo que se considera muy improbable.

Ejemplo: contador de 2 bits

¿Path$(\varnothing, M, 4)$ devuelve *sí*?

$O_1 = (\varnothing, \{c_1\}, \varnothing)$ y $O_2 = (\{c_1\}, \{c_2\}, \{c_1\})$, con $S_0 = \varnothing$ y $M = \{c_1, c_2\}$.



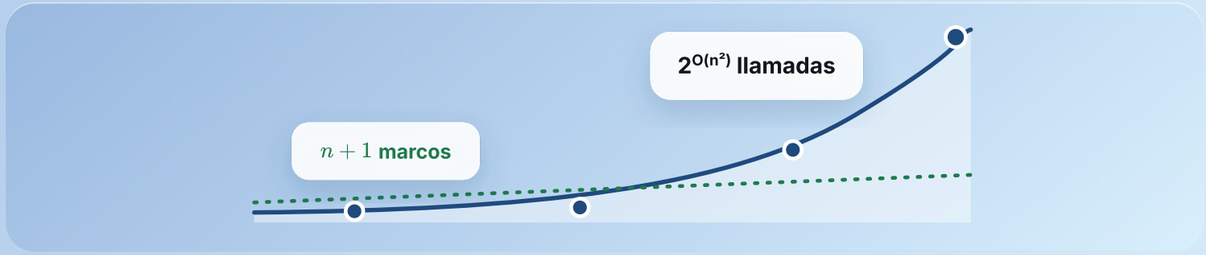
La grafica que el algoritmo nunca construye

Plan mínimo de 3 pasos: $\varnothing \to \{c_1\} \to \{c_2\} \to \{c_1, c_2\}$.

S'	$x = \text{Path}(\varnothing, S', 2)$	$y = \text{Path}(S', M, 2)$	Resultado
$\varnothing$	sí	no	se descarta
$\{c_1\}$	sí	sí	**sí ✓**

Se detiene en $\{c_1\}$: 20 llamadas en total y nunca más de 3 marcos en la pila.

¿Y si n crece?



20

$n = 2$

llamadas a Path · plan de 3 pasos · 3 marcos.

454

$n = 3$

llamadas a Path · plan de 7 pasos · 4 marcos.

48 795

$n = 4$

llamadas a Path · plan de 15 pasos · 5 marcos.

23 167 425

$n = 5$

llamadas · plan de 31 pasos · 6 marcos. Con $n = 60$ bastarían 61 marcos.

Búsqueda explícita

$2^{O(n)}$

- Construye la grafica y aplica BFS o DFS.
- Es el más rápido: polinomial en el tamaño de la grafica.
- Inviable en memoria para n grande.

Savitch · Path

Polinomial

- $O\big(n(n+\log k)\big)$ bits de memoria.
- Tiempo $2^{O(n^2)}$: recalcula todo.
- Demuestra que Planning $\in$ PSPACE.

Planificadores prácticos

Heurístico

- FF (Hoffmann y Nebel, 2001) y Fast Downward (Helmert, 2006).
- Planeación como SAT (Kautz y Selman, 1992).
- Sin garantía en el peor caso.

Enfoques para resolverlo

Explorar la grafica es rápido pero exige memoria exponencial; Path ahorra memoria a costa de tiempo; en la práctica se usan heurísticas y la estructura de cada dominio.



APLICACIÓN · INSTANCIA FICTICIA

Vuelos con escalas y visas

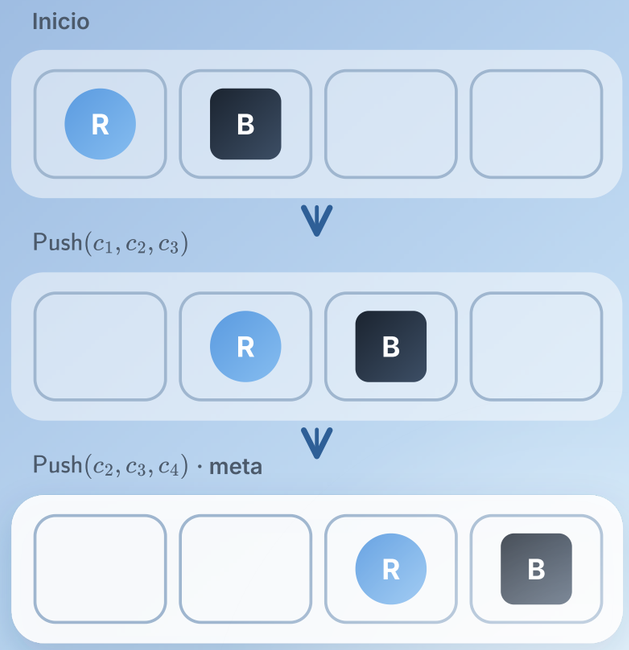
$$S_0 = \{c_1, c_6\} \;\rightarrow\; S^* = \{c_5, c_6, c_7, c_8\}$$

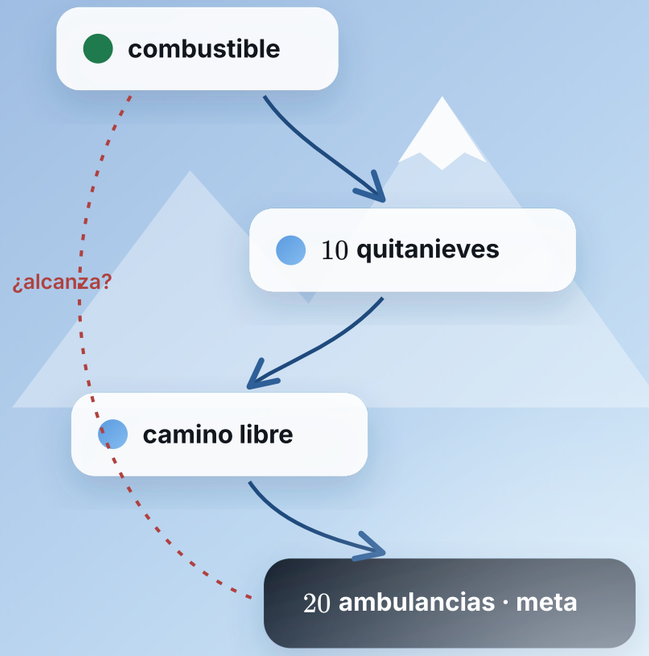
Para volar a Tokio hace falta la visa japonesa, que solo se tramita en París. El viajero pasa dos veces por Madrid, pero en **estados distintos**: en S_3 aún no tiene la visa y O_7 no es aplicable.

Plan mínimo de 7 acciones. Path lo verifica con $L = 8$ y nunca más de 4 marcos.

¿Dónde aparece?

Problemas donde cada acción cambia las acciones disponibles.





Robots de almacén

"Como Sokoban, PSPACE-completo en general (Culberson, 1997). Con 4 celdas: $n = 12$ condiciones R@c, B@c, F@c y $k = 10$ operadores Move y Push."

Logística y socorro

"Ambulancias que necesitan quitanieves, que necesitan combustible... También rompecabezas y misiones espaciales como el Remote Agent de la NASA."

Longitud acotada

NP

- Se pide un plan de a lo más $q(n)$ pasos.
- El plan es un certificado polinomial.
- Se verifica simulando los operadores.

Restricciones · Bylander

PSPACE-c

- Aun con 2 prerrequisitos y 2 postcondiciones.
- Caso tratable: prerrequisitos positivos y 1 postcondición.
- Hacen falta restricciones muy severas.

Primer orden

¿Decidible?

- Predicados y variables en lugar de proposiciones.
- Depende de símbolos de función y eliminaciones.
- Erol, Nau y Subrahmanian (1995).

Casos especiales

QSAT y los juegos de dos jugadores comparten la idea: se resuelven en espacio polinomial reutilizando memoria y guardando un bit por subproblema.

Referencias

Fuente principal: Kleinberg y Tardos, *Algorithm Design*, capítulo 9.

Bylander, T. (1994). The computational complexity of propositional STRIPS planning. *Artificial Intelligence, 69*(1–2), 165–204.

Culberson, J. (1997). *Sokoban is PSPACE-complete* (TR 97-02). University of Alberta.

Edelkamp, S., Fink, J., Gregor, P., Jonsson, A. y Nebel, B. (2026). *Intermediate results on the complexity of* STRIPS_1^1. arXiv:2602.08708.

Erol, K., Nau, D. S. y Subrahmanian, V. S. (1995). Complexity, decidability and undecidability results for domain-independent planning. *Artificial Intelligence, 76*, 75–88.

Fikes, R. E. y Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence, 2*, 189–208.

Ghallab, M., Nau, D. y Traverso, P. (2004). *Automated planning: Theory and practice*. Morgan Kaufmann.

Helmert, M. (2006). The Fast Downward planning system. *JAIR, 26*, 191–246.

Hoffmann, J. y Nebel, B. (2001). The FF planning system. *JAIR, 14*, 253–302.

Hopcroft, J. E., Schwartz, J. T. y Sharir, M. (1984). PSPACE-hardness of the “warehouseman’s problem”. *IJRR, 3*(4), 76–88.

Kautz, H. y Selman, B. (1992). Planning as satisfiability. *ECAI-92*, 359–363.

Kleinberg, J. y Tardos, É. (2005). *Algorithm design*. Pearson.

Muscettola, N., Nayak, P. P., Pell, B. y Williams, B. C. (1998). Remote Agent: To boldly go where no AI system has gone before. *Artificial Intelligence, 103*, 5–47.

Savitch, W. J. (1970). Relationships between nondeterministic and deterministic tape complexities. *JCSS, 4*(2), 177–192.

Sipser, M. (2013). *Introduction to the theory of computation* (3.ª ed.). Cengage.

Stockmeyer, L. J. y Meyer, A. R. (1973). Word problems requiring exponential time. *STOC ’73*, 1–9.

PLANNING

¿Preguntas?

$S_0 \to \cdots \to S^*$ en espacio polinomial

resultado
PSPACE-c

Gracias por su atención

La memoria es el recurso que se recicla: Planning se decide en espacio polinomial aunque sus planes sean exponenciales.

Equipo

- Jesús Antonio Romero Godoy · 321144292
- Alan Alexzander González Murga · 320079629
- Complejidad Computacional · UNAM
- 18 de noviembre de 2026